

The Alcove & Adder Programming Manual

Alcove Team

The Alcove & Adder Programming Manual: A Deep Dive into High-Performance Lisp and Python-Like Dialect VM

Welcome to the definitive guide to **Alcove** and **Adder**. This book is built programmatically: every single code block is executed by the actual compiled **alcove** and **adder** binaries, verifying both syntax and runtime output in real-time.

Chapter 1: Introduction & Architecture

1.1 One VM, Two Dialects

Alcove and Adder are two complementary frontends compiling to the same register-based bytecode virtual machine. * **Alcove** represents the classic Lisp-style S-expression syntax. * **Adder** is a Python-inspired block-based syntax that compiles down to the exact same S-expressions and runs on the same VM.

Because they compile to the same underlying S-expressions, they share the same global namespace, the same FFI bindings, the same garbage collection behaviors, and can transparently import (**require**) and call each other.

1.2 Pointer Tagging Layout

On 64-bit platforms, allocations returned by **malloc** are aligned to 8 bytes, which leaves the lowest 3 bits of a pointer address as 000. Alcove uses this alignment to store immediate values (fixnums and characters) directly inside the pointer variable (**exp_t ***) itself, skipping the heap: * 000 (**TAG_PTR**): Standard heap-allocated node (e.g. lists, dicts, vectors). * 001 (**TAG_FIX**): 61-bit signed immediate integer. * 010 (**TAG_CHAR**): Unicode character immediate.

This immediate value tagging avoids memory allocation, refcount tracking, and collection overhead.

1.3 Memory Management & Teardown

Alcove uses a fast-path reference counting model. To handle cycles formed by local closures capturing their parent frames, the VM implements a cycle-breaking pass (`env_break_self_cycle`) on environment destruction. For user-defined structures, (`heap-stats`) is provided to track live allocation cells.

Chapter 2: Data Types & Collections

2.1 Numbers and Characters

Alcove features fixnums, floats, ratios (exact fractions), and decimals.

Alcove (Lisp)

```
In [1]: Out [1]: 3
```

```
In [2]: Out [2]: 0
```

```
In [3]: Out [3]: t
```

```
In [4]: Out [4]: 1.25m
```

```
In [5]: Out [5]: #\A
```

```
In [6]: Out [6]: t
```

Adder

```
In [1]: Out [1]: 3
```

```
In [2]: Out [2]: 1/3
```

```
In [3]: Out [3]: t
```

```
In [4]: Out [4]: 1.25m
```

```
In [5]: Out [5]: #\A
```

```
In [6]: Out [6]: t
```

2.2 Collections

Alcove provides multiple native collections: * **Lists**: Linked cons cells. * **Vectors**: Flat arrays with double-ended queue operations. * **Hash Maps**: Mutable hash

tables. * **Persistent Maps**: Immutable Hash Array Mapped Tries (**hamt**). *
Blobs: Byte arrays.

Alcove (Lisp)

```
In [1]:Out[1]:(1 2 3)
```

```
In [2]:Out[2]:#[20 20 20 20 20 20 20 20 20 20]
```

```
In [3]:Out[3]:{"version" 2, "name" "alcove"}
```

```
In [4]:Out[4]:{"b" 2, "a" 1}
```

```
In [5]:Out[5]:#b"hello"
```

Adder

```
In [1]:Out[1]:(1 2 3)
```

```
In [2]:Out[2]:#[20 20 20 20 20 20 20 20 20 20]
```

```
In [3]:Out[3]:{"version" 2, "name" "adder"}
```

```
In [4]:Out[4]:{"b" 2, "a" 1}
```

```
In [5]:Out[5]:#b"hello"
```

Chapter 3: Control Flow & Pattern Matching

3.1 Local Bindings & Assignment

We can bind variables locally using `let` or `=` assignment.

Alcove (Lisp)

```
In [1]:Out[1]:30
```

Adder

```
In [1]:Out[1]:30
```

3.2 Conditionals & Pattern Matching

We can use `if`, `cond`, `case`, and structural pattern matching `match`.

Alcove (Lisp)

```
In [1]:Out[1]: "yes"
```

```
In [2]:Out[2]: 3
```

Adder

```
In [1]:Out[1]: "yes"
```

```
In [2]:Out[2]: 3
```

3.3 Loops

Supported loop constructs include `while`, `repeat`, `for`, and `each`.

Alcove (Lisp)

```
In [1]:0  
1  
2  
Out[1]:nil
```

Adder

```
In [1]:Out[1]:nil
```

Chapter 4: Functions & Polymorphism

4.1 Functions and Multi-arity

Functions are defined via `def` (single-arity) or `defn` (multi-arity).

Alcove (Lisp)

```
In [1]:Out[1]:#<procedure:area@...>
```

```
In [2]:Out[2]:78.5
```

```
In [3]:Out[3]:20
```

Adder

```
In [1]:Out[1]:#<procedure:area@...>
```

```
In [2]:Out[2]:78.5
```

```
In [3]:Out[3]:20
```

4.2 Escape Continuations

Continuations can be captured via `call/cc` or early returns via `defc`.

Alcove (Lisp)

```
In [1]:Out[1]:#<procedure:search@...>
```

```
In [2]:Out[2]:t
```

Adder

```
In [1]:Out[1]:#<procedure:search@...>
```

```
In [2]:Out[2]:nil
```

4.3 Macros

Macros are defined using `defmacro`. They manipulate S-expressions and expand at compile time.

Alcove (Lisp)

```
In [1]:Out[1]:#<macro:when@...>
```

```
In [2]:Out[2]:(when t (prn "hello"))
```

Adder

```
In [1]:Out[1]:#<macro:when@...>
```

```
In [2]:Out[2]:(when t (prn "hello"))
```

4.4 Structs & Multimethods

Records are defined via `defstruct`. Custom type polymorphism is supported via `defmulti` and `defmethod`.

Alcove (Lisp)

```
In [1]:Out[1]:point
```

```
In [2]:Out[2]:{"__type__" point, "x" 3, "y" 4}
```

```
In [3]:Out[3]:3
```

```
In [4]:Out[4]:t
In [5]:Out[5]:greet
In [6]:Out[6]:{"point" #<procedure@...>}
In [7]:Out[7]:{"int" #<procedure@...>, "point" #<procedure@...>}
In [8]:Out[8]:"hello point!"
In [9]:Out[9]:"hello int!"
```

Adder

```
In [1]:Out[1]:point
In [2]:Out[2]:{"__type__" point, "x" 3, "y" 4}
In [3]:Out[3]:3
In [4]:Out[4]:t
In [5]:Out[5]:greet
In [6]:Out[6]:{"point" #<procedure@...>}
In [7]:Out[7]:{"int" #<procedure@...>, "point" #<procedure@...>}
In [8]:Out[8]:"hello point!"
In [9]:Out[9]:"hello int!"
```

Chapter 5: Advanced Systems

5.1 Stateful Generators

Generators provide stateful yield-style lazy sequence iteration.

Alcove (Lisp)

```
In [1]:Out[1]:{"gs" 1, "gk" 1, "gc" 1, "ge" 5}
In [2]:Out[2]:1
In [3]:Out[3]:2
```

Adder

```
In [1]:Out[1]:{"gs" 1, "gk" 1, "gc" 1, "ge" 5}
```

```
In [2]:Out[2]:1
```

```
In [3]:Out[3]:2
```

5.2 JSON Serialization

JSON encoding and decoding is built-in.

Alcove (Lisp)

```
In [1]:Out[1]:{"score":99,"name":"alcove}"
```

```
In [2]:Out[2]:{"score" 99, "name" "alcove"}
```

Adder

```
In [1]:Out[1]:{"score":99,"name":"adder}"
```

```
In [2]:Out[2]:{"score" 99, "name" "adder"}
```